UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA
Faculty of Engineering, Built Environment and Information Technology

EAI 320

Intelligent Systems

Practical 2

Name and Surname	Student Number	Signature	% Contribution
J. Venter	19002409		50
A. van Niekerk	19007583		50

By signing this assignment we confirm that we have read and are aware of the University of Pretoria's policy on academic dishonesty and plagiarism and we declare that the work submitted in this assignment is our own as delimited by the mentioned policies. We explicitly declare that no parts of this assignment have been copied from current or previous students' work or any other sources (including the internet), whether copyrighted or not. We understand that we will be subjected to disciplinary actions should it be found that the work we submit here does not comply with the said policies.

May 18, 2021

Contents

1 Introduction 1
2 Background Theory 2
  2.1 Initialization 3
  2.2 Selection 3
  2.3 Crossover 3
  2.4 Mutation 3
  2.5 Solution Check 3
3 Method 4
  3.1 Assumptions 4
  3.2 The Genetic Algorithm 4
4 Results 6
5 Discussion 11
6 Conclusion 13

List of Figures

2.0.1 flowchart of the Operation of a GA 2
4.0.1 Data 1 Avg and max fitness over generations 6
4.0.2 Data 2 Avg and max fitness over generations 6
4.0.3 Data 1 Avg and max fitness over generations with a cull rate of 0.1 % 7
4.0.4 Data 2 Avg and max fitness over generations with a cull rate of 0.1 % 7
4.0.5 Data 2: Fitness score of each generation for population sizes 10,100,500 8
4.0.6 Data 2: A 8
4.0.7 Data 2: average and max fitness over generations with a crossover rate of 90% . 9
4.0.8 Data 2: average and max fitness over generations with a crossover rate of 10% . 9
4.0.9 Data 2: average and max fitness over generations with a crossover rate of 90% initialization state is of worst case 10
4.0.10 Data 2: average and max fitness over generations with a crossover rate of 10% initialization state is of worst case 10

1 Introduction

The goal of this Practical is to construct a Genetic Algorithm that will initialize itself with randomly selected values of "R","P" or "S" for each gene in the algorithm. Each chromosome needs to be populated to a length of 81. A population of specified size is then made with a number of chromosomes. This algorithm then evolves to a specified number of generations and evolves intending to achieve a specified optimal solution derived from a set of CSV data files

2 Background Theory

The genetic algorithm is a heuristic search algorithm that was inspired by Charles Darwin in 1859 and was introduced by John Holland in 1975. A Genetic Algorithm, GA, relies on the theory of evolution and strives to achieve a natural selection for evolution.[1]

A GA is comprised out of a fitness function and an initial population, The goal of a GA is to find an optimal solution. This optimal solution is named the fitness function and can be determined by anything. The GA then needs to evolve as best as possible while using natural selection to obtain or to get as close as possible to this Optimal solution, To obtain this the GA operates in a series of steps. By using randomness to ensure natural selection the GA can achieve its optimal solution by using natural selection. These steps are as follows and will be explained briefly and a graphical representation of them can be seen in Figure 2.0.1.

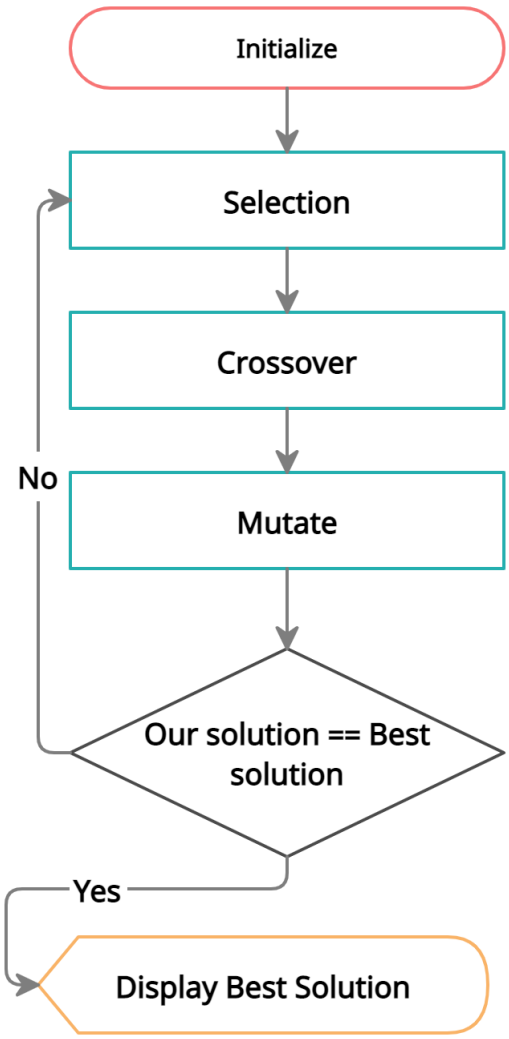

Figure 2.0.1: flowchart of the Operation of a GA

2.1 Initialization

The first step of a GA is to initialize its population and specify its gene size. The size of the population can be determined by the user using the GA, a parameter of how many generations to be created must also be inputted by the user. The population is initialized by creating a chromosome consisting out of a series of genes, the chromosome is of specified length determined by the fitness function, for the initialization process the chromosomes are populated with genes that have been selected randomly, in the case of Rock Paper Scissors a gene is a random choice of "R","P" or "S".

2.2 Selection

The selection process is what distinguishes GA's from one another, there are many selection algorithms and the main idea of a selection algorithm is to implement a natural selection process, by implementing randomness into the selection function. The goal of the selection function is to simulate a natural selection and use this natural aspect to select 2 chromosomes randomly/-naturally that will be crossover-ed or mutated.

2.3 Crossover

There are various ways to implement the crossover function,but the goal of the crossover function is to evolve, evolution is achieved by taking part of one chromosome and another part of another chromosome to formulate a new chromosome in the hopes that the new chromosome is a better fit for the solution. The Crossover function receives two chromosomes from the selection function and takes a random part of Chromosome one and a random part of chromosome two, to still match the given length and formulate two new chromosomes that is used to populate the next generation.

2.4 Mutation

The chromosomes that are not used by the crossover function is passed on to the mutation function, the goal of the mutation function is to implement randomness and to hopefully create/generate a better chromosome than the one passed in. The mutate function chooses a random position(s) in the chromosome and changes it to a random value/object.

2.5 Solution Check

By using all of the above methods the GA evolves closer to the optimal solution, after each generation has been created the fitness function is used to check if the GA has found the optimal solution. If it has been found the goal has been achieved and the program stops. If the goal has not been achieved the new generation is passed into the selection function and the process starts all over again until the end of the specified generations have been achieved or the optimal solution has been found.

3 Method

3.1 Assumptions

From the files provided there are 2 csv data files, called data1.csv and data2.csv. In these files there is a History and a played solution, The history is of length 4 and the solution is of length one, each data file consists out of a million lines with a history and a solution, bear in mind that there may be more that there may be different solutions played for the same history, the optimal solution for that history is then considered the solution that was played the most. If there is no record of a history the the solution of that particular history is considered a don't care value. For Data 1 there are 45 histories that have a optimal solution assigned and 36 histories that have a don't care value. For Data 2 there are 81 unique optimal solutions for each History. For the Final solution generated by the GA, the moves are in the order of histories comprised from the "DFS" Algorithm "RRRR" to "SSSS".

3.2 The Genetic Algorithm

The Genetic Algorithm was constructed with the same layout as described in Background Theory on page 2. The same order of functions was used.

Fitness Algorithm

The fitness algorithm must be determined from the given data.csv files. The optimal solution will be found from these files. The optimal solution is determined by iterating through the entire csv file. The algorithm compares the history of the csv file to the history in the directory file, which is in the correct order. If the histories match then the solution is looked at, if it is "R" the R variable is increased by one, if it is "P" the P variable is increased by one and the same for S, this is done for each element in the data file. The final optimal solution for each history is then selected as the variable with the highest number. After the Optimal solution is derived from the data files. There needs to be a function that compares the fitness function with a possible solution from the GA, this CheckFitness function compares each gene in the fitness solution to each gene in the GA solution and if the genes match the fitness value of the GA solution is incremented by one. Therefore the maximum achievable fitness value for the optimal solution from the data1.csv file is a size of 45, while the maximum achievable fitness value for the optimal solution from the data2.csv file is a size of 81.

Selection Algorithm

For the selection process there are various methods to chose from,the favourites are the roulette search, tournament selection, stochastic selection and ranked selection [2]. The selection chosen for the GA is the Roulette selection. The roulette selection operates by assigning a probability value to each gene proportionate to its fitness, because the relationship of probability to fitness is linear, when the GA is formed with a population size of 1000 the value of probabilities are so small they are almost identical, causing the GA to go to a stagnant phase where no evolution occurs or very slow improvement, even no improvement in some cases. Because of this the

Ranked selection was integrated into the roulette selection. This was done by creating a edge for the chromosomes with a higher fitness value, instead of using a linear relationships of probability, exponential relationships was used. This was done by raising 2 to the power of the fitness value of the chromosome,if the fitness value of the chromosome is equal to f and the sum of all the fitness values are fs therefore the probability of the chromosome being chosen can be written as:

$$\frac{2^f}{fs} \tag{1.1}$$

The cumulative is then added list of all the probabilities and then a random value is selected and that particular chromosome is added to the crossover list in the order they will need to be crossover-ed.

Crossover Algorithm

The crossover functions receives a reorganised selected list from the selection algorithm. There is a specified crossover rate from 0 to 1 where it represents the percentage of the population that should be crossover-ed, the rest will mutated. The main structure of the crossover is: Two parents are selected from the top of the selection list generated by the selection algorithm, a random value between 0 and 1 is chosen, this represents the crossover relationship from

$$Parent1 : Parent2$$

in the form of

$$trunc(r * 81) : 81 - trunc(r * 81)$$

Where r is a random value between 0 to 1 from each pair of parents two new children are generated where the first child consists of

$$Parent1 + Parent2$$

and the second child consists of

$$Parent2 + Parent1$$

in the respective relationship calculated from the random value r. These new chromosomes are passed into the new generation, and then the next two chromosomes of the current generation is selected and the process repeats up until the specified crossover percentage of the current generation has been achieved. The rest of the generation is sent to the mutate process.

Mutation Algorithm

The mutation process is used for when the growth comes to a halt, this is implemented by selecting a random position in the chromosomes that have not been crossover-ed and this random position is changed to a random selection of “R”,“P” or “S”

After all of the above processes have been completed the new generation is checked by the CheckFitness function and if a solution of max achievable fitness value is found the GA stops and returns that particular solution. If the optimal solution is not found withing the new generation the new generation becomes the current generation and a new generation is obtained by the process explained above. This iterates until a solution is found or the specified generations are reached.

4 Results

This section is dedicated to interpreting the data from various graphs to determine the effect that each elements has on the GA.There was various simulations that were ran to answer the questions that may arise, questions including "At which population is the optimal solution achieved the fastest?" or "How much generations do we need for a specific population to achieve the optimal solution?"

In figures 4.0.1 and 4.0.2 as discussed in Fitness Algorithm on 4, Data 1 has lower achievable fitness value because of the 36 don't care values, while data 2 has 81 unique solutions. It can be directly deducted that the GA takes longer to achieve the fitness function with a higher unique solution from figure 4.0.1 and 4.0.2 below.

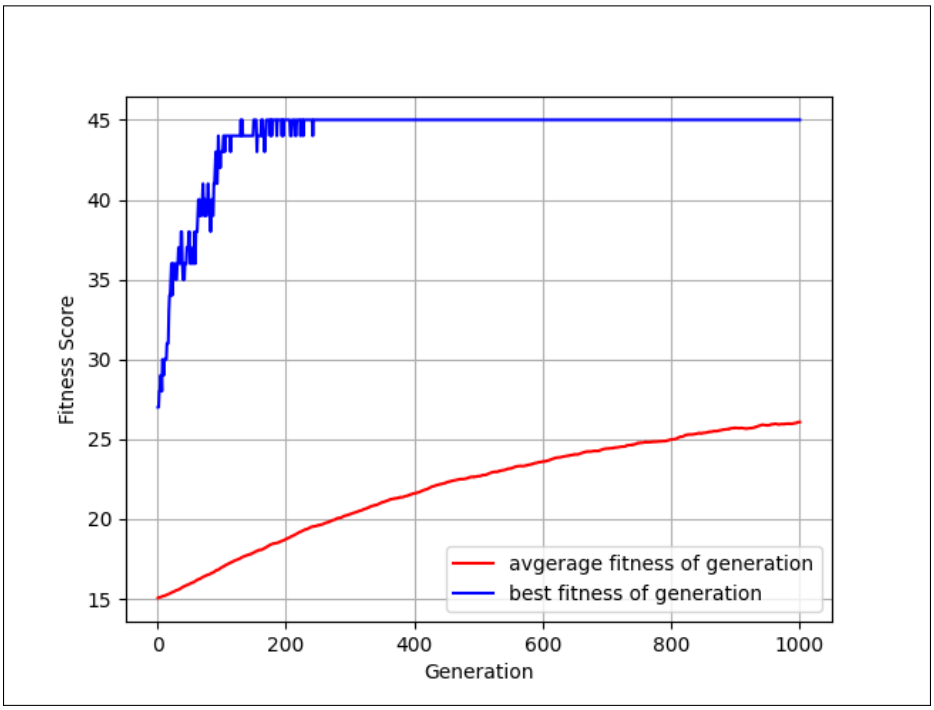

Figure 4.0.1: Data 1 Avg and max fitness over generations

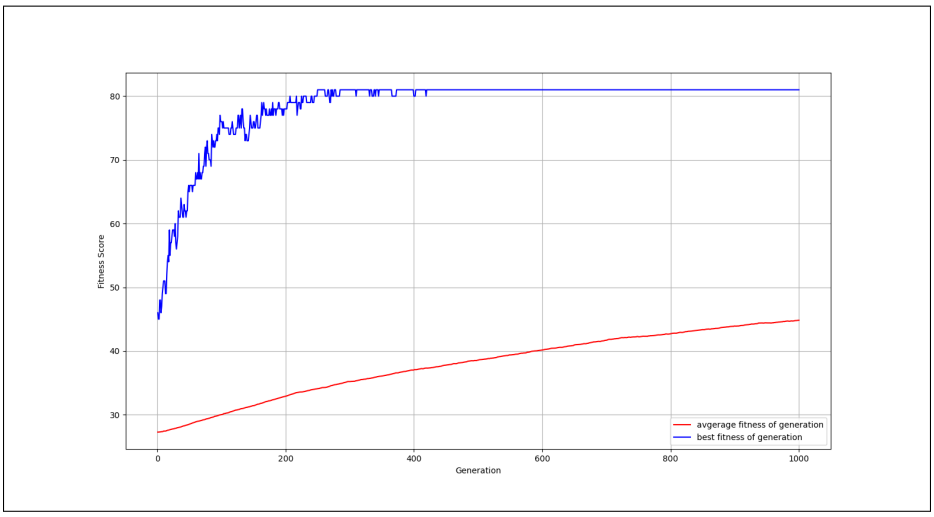

Figure 4.0.2: Data 2 Avg and max fitness over generations

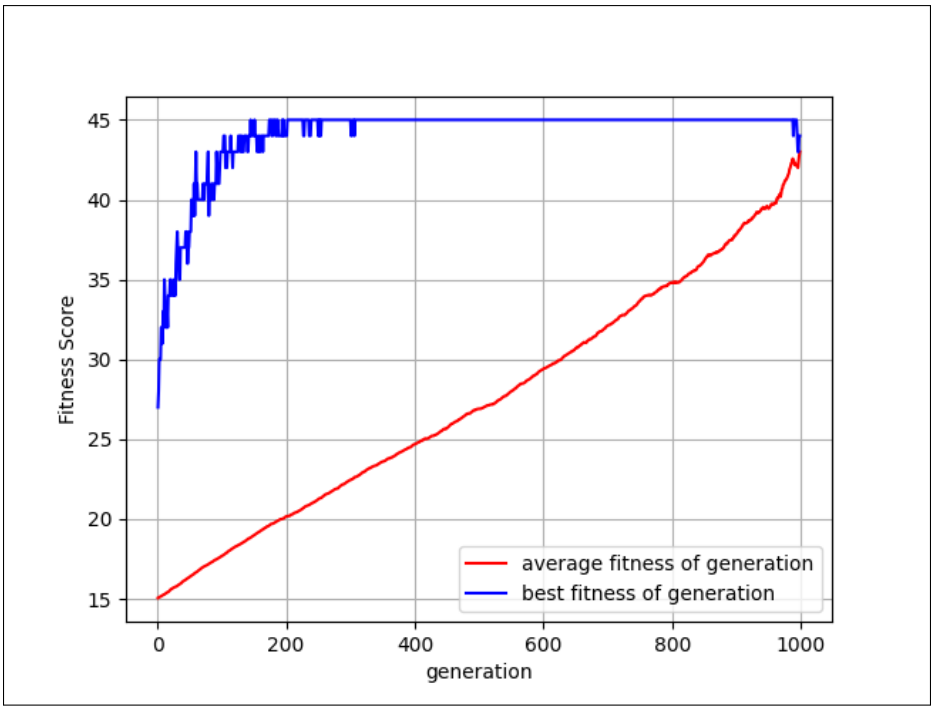

Figure 4.0.3: Data 1 Avg and max fitness over generations with a cull rate of 0.1 %

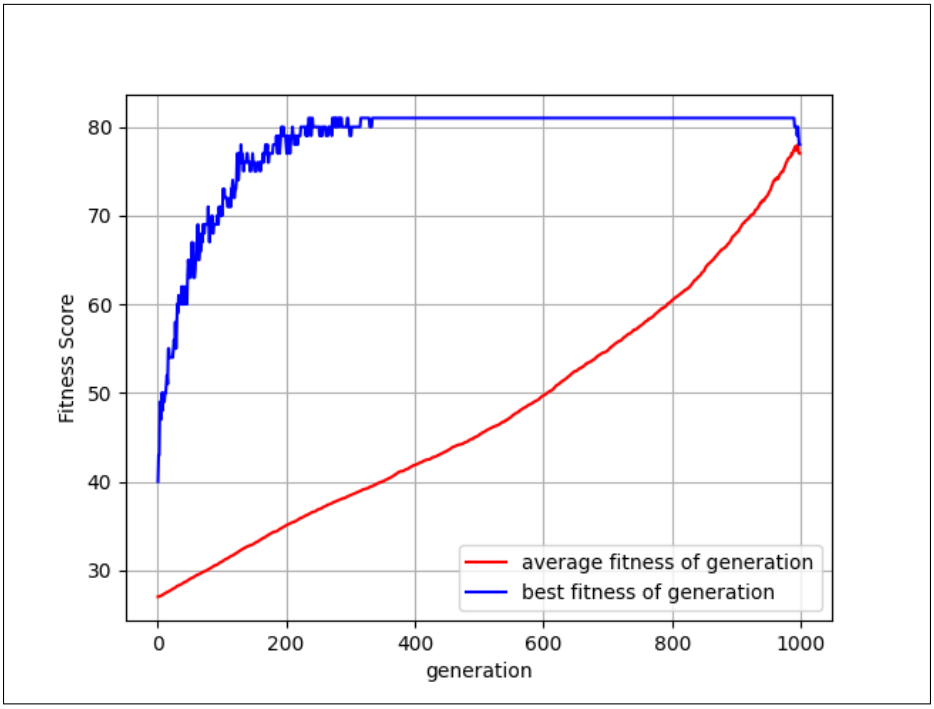

Figure 4.0.4: Data 2 Avg and max fitness over generations with a cull rate of 0.1 %

The following figures was simulated for the data 2 optimal solution,to determine the best optimal values to obtain the optimal solution the fastest and most efficiently

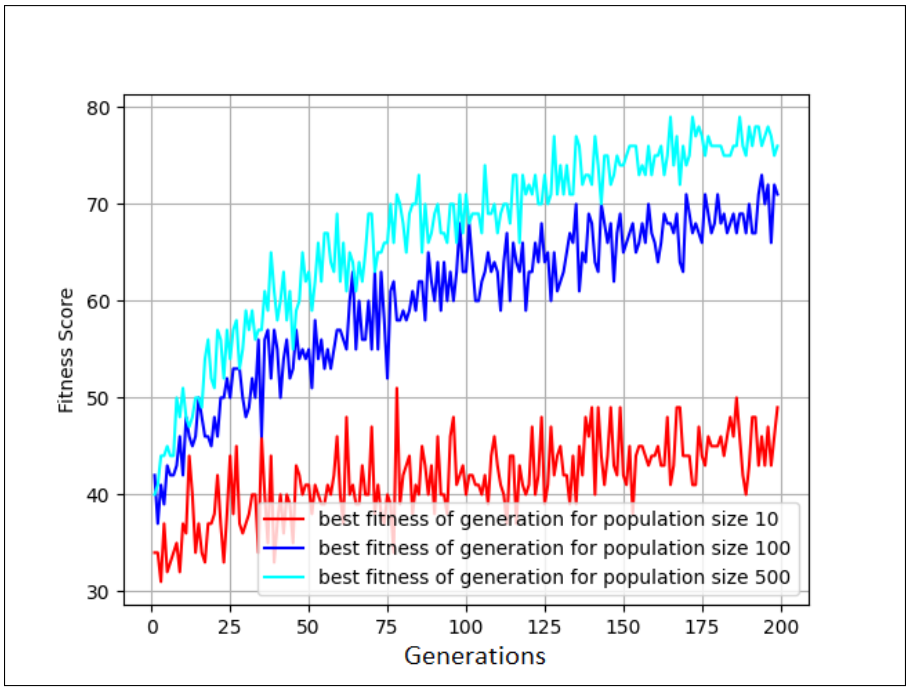

Figure 4.0.5: Data 2: Fitness score of each generation for population sizes 10,100,500

As can be seen from figure 4.0.5, higher population size leads to less spread in the results. The population size also has an effect on how fast the fitness function is found in a given amount of generations.

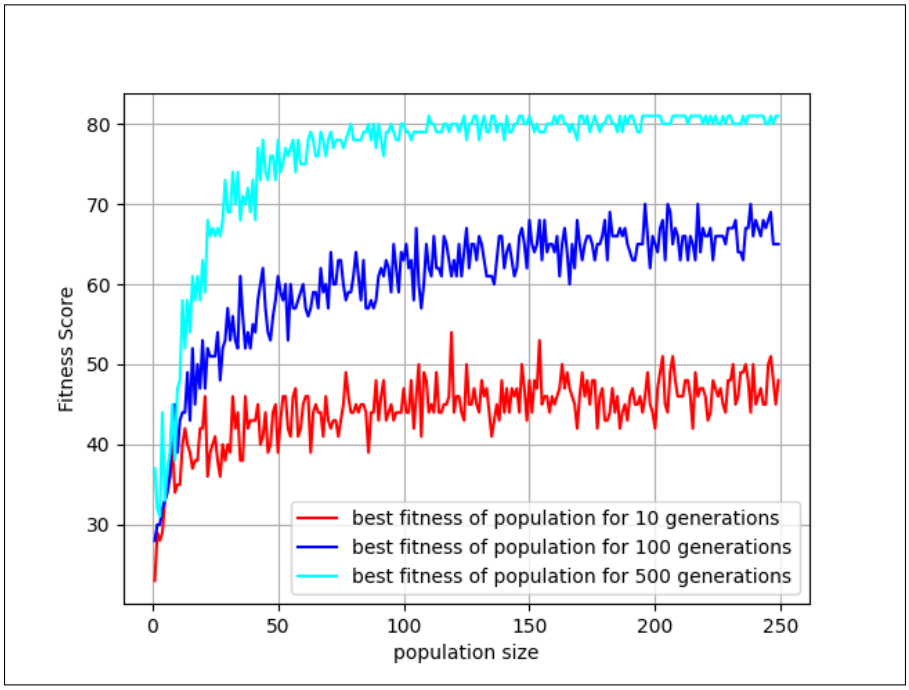

Figure 4.0.6: Data 2: A

From figure 4.0.6 it can bee seen that the increase in generations has a large effect on the time it takes to get the fitness function. Higher generations also leads to less spread in the results.

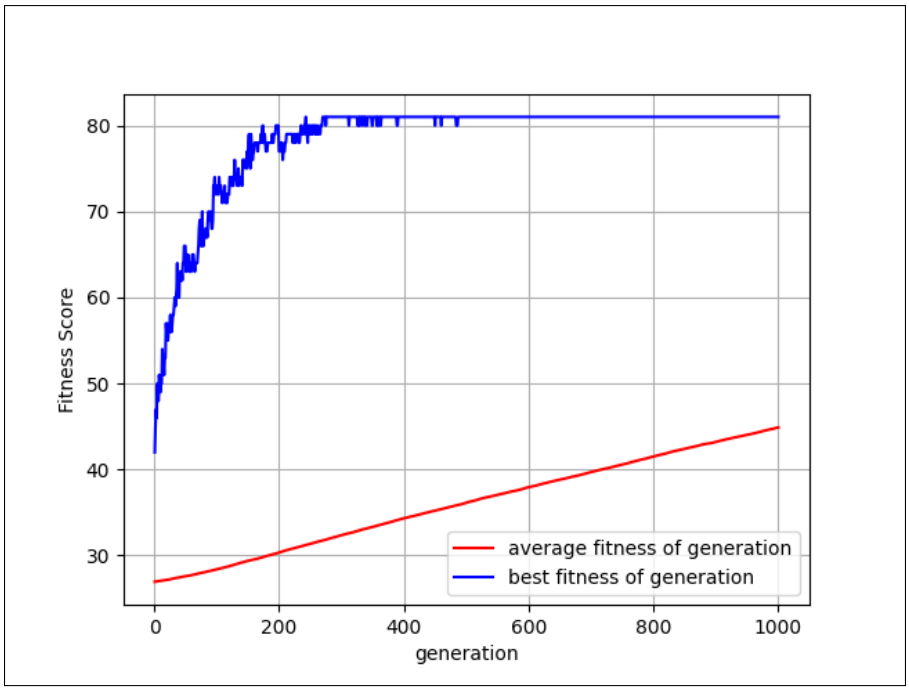

Figure 4.0.7: Data 2: average and max fitness over generations with a crossover rate of 90%

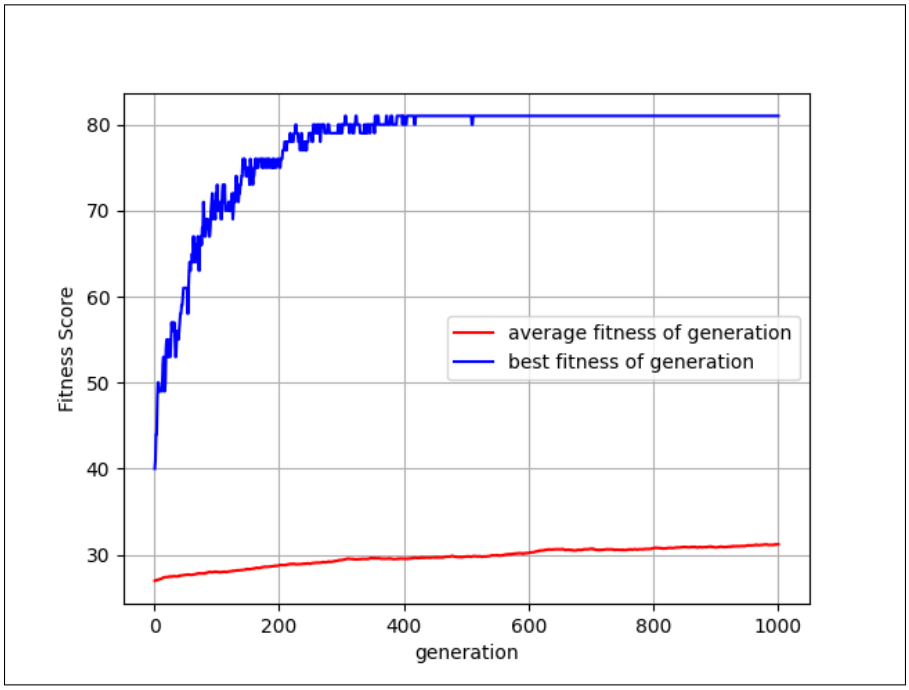

Figure 4.0.8: Data 2: average and max fitness over generations with a crossover rate of 10%

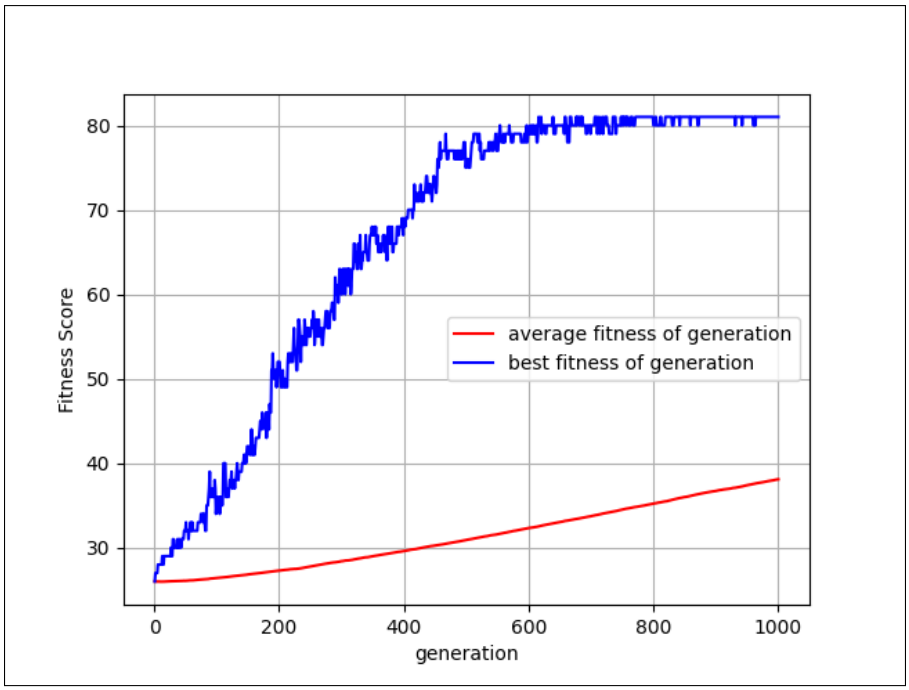

Figure 4.0.9: Data 2: average and max fitness over generations with a crossover rate of 90% initialization state is of worst case

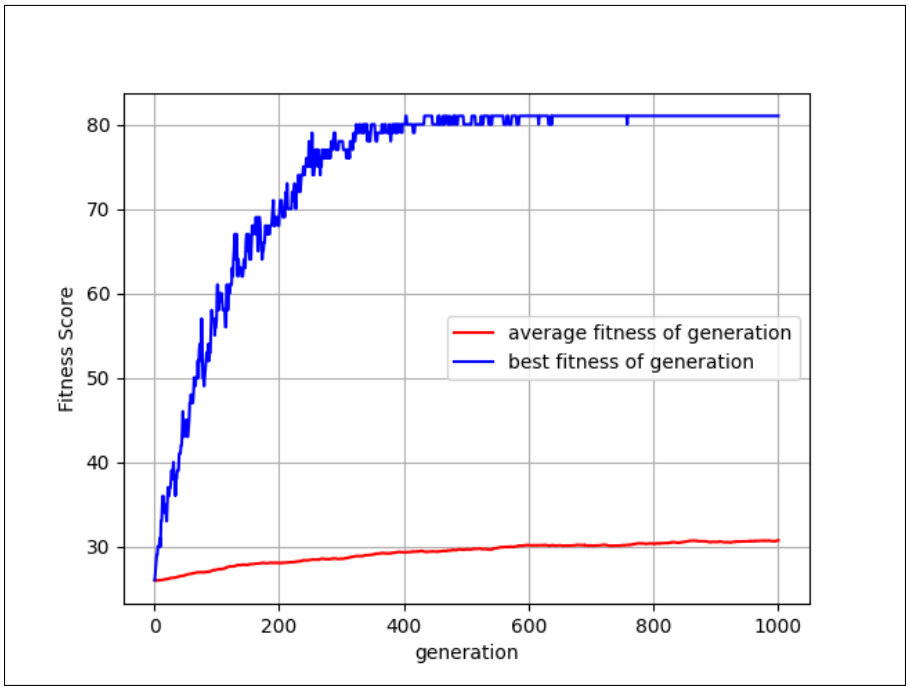

Figure 4.0.10: Data 2: average and max fitness over generations with a crossover rate of 10% initialization state is of worst case

From figures 4.0.9 and 4.0.10 the first generation was initialize with only "R"'s this was is done to see the effect of mutation on a worst case scenario.

5 Discussion

From the results obtained by the GA in Results, a lot can be seen. From figures 4.0.1 and 4.0.2 one can clearly see that the GA finds a optimal solution for the set of Data 1 before 200 generations have passed, but for data 2 it obtains a optimal solution just before 400 generations, almost 2 times the generation count for data 1's solution, bearing in mind that both use the same GA and both population sizes are of size 1000, the data 1 solution was obtain much faster. This is because the fitness function obtained from data 1 only has 45 optimal genes, as discussed in Method the other 36 genes are don't cares, i.o.w. they're not of importance to the GA, because of this there is less chance for the GA to have the incorrect sequence, while the data 2 fitness function has 81 unique genes meaning the GA must get all 81 genes correct for data 2, resulting in more generations needed to conclude a optimal solution, where the data 1 solution is only 45 unique genes and the don't cares can be anything, meaning only 45 genes have to match exactly, almost half the amount of genes that must match in data 2.

From figures 4.0.1 and 4.0.2 the red line represents the average fitness value of each generation, the goal of a GA is to grow and to evolve, it evolves by moving closer to the optimal solution that satisfies the fitness function, The average fitness value after each generation is supposed to increase when comparing it to this analogy, it can be see that the red line is constantly increasing in average fitness value meaning that the new generations average fitness value is greater then the previous generations' average fitness value, from this one can conclude that the GA truly evolves and improves on itself from the previous generations.

As seen in Results in figures 4.0.5 and 4.0.6 on page 8 it can bee seen that both population size and total generations have an effect on the data spread and the time it takes to obtain the optimal solution. It can be deducted that the total generations has an larger effect on the results than population size, since the slope of the graph in 4.0.6 is much steeper and reaches the optimal solution faster with increasing generations versus the graph of increasing population sizes in 4.0.5. Hence wanting more accuracy and a faster compilation time it is more important to increase the number generations than the population size.

From figures 4.0.7 and 4.0.8 and 4.0.9 and 4.0.10, there was played with the initial state, as well as the crossover and mutation rates. for 4.0.7 the Generations were 1000 and the population size was also 1000 and the same parameters was used for figures: 4.0.8, 4.0.9, 4.0.10 we can clearly see from this what the importance is of the mutation rate,we can see that the mutation function of this GA does exactly what it is intended to do and comes in very handy in this particular case. For 4.0.9 and 4.0.10 the worst possible initialization was chosen and the chromosomes only consisted of "R"'s, if not for the mutation rate in this particular case the best optimal solution would not have been found. The mutation function changes specific random genes in each chromosome and in this particular case where the entire generation consists of only "R"'s, this is the best possible way to improve the solution of each generation, however when the case is not the worst case the higher crossover rate GA dominates the lower crossover rate GA and find the best optimal solution almost a 100 generations earlier, see figures 4.0.7 and 4.0.8.

For figures 4.0.3 and 4.0.4 a cull rate was applied, the cull rate is set at 0.1 % and it takes away that percentage of the generation after each iteration. What we can see from this is that the average fitness score of each generation improves dramatically as the end of the generation cycle is approached, this is only because the culling takes away the worst possible solution, because we lose the worst solution of each generation, at the end of the cycle we are only left with a few chromosomes each with a high fitness score.Therefore it appears like the solution is

obtained faster, but in fact it is not. The GA only improves in efficiency but not in the amount of generations to determining the best solution.

6 Conclusion

The goal of this assignment was to construct a Genetic Algorithm that will be able to find a optimal solution after given a fitness function, the Genetic Algorithm was to implemented in a Rock,Paper, Scissor structure. The fitness value was obtained successfully from the data files, both for data 1 and for data 2. The Genetic Algorithm was also constructed with a set of required functions to be able to evolve and improve.

From the Results on page 6 and from Discussion on page 11, one can conclude that the GA was implemented successfully and obtained the results expected from it.It can also be concluded that the GA truly finds the optimal solution and it is identical to the solution found from the Data files, It can also be concluded that the GA does improve from previous generations and builds on its-self.

Bibliography

[1] D. Simoncini and K. Y. Zhang, "Population-based sampling and fragment-based de novo protein structure prediction," Sep 2018. [Online]. Available: https://www.sciencedirect.com/science/article/pii/B9780128096338205074

[2] "Genetic algorithms - parent selection." [Online]. Available: https://www.tutorialspoint.com/genetic_algorithms/genetic_algorithms_parent_selection.htm